Industrial Vision

Lecture 7 - SIFT

+ board

Where we are

In the last lecture we constructed **scale space**. The image is convolved with scale-normalized LoG filters $\sigma^2 \nabla^2 G_\sigma$ over many values of σ, and the extrema over **space and scale together** are selected. Each keypoint was obtained as a triple (x, y, σ), corresponding to a blob of radius $r = \sqrt{2}\,\sigma$.

Two problems remain.

1. The LoG is **expensive to compute**, since a new and progressively larger kernel is required at every scale.
2. An extremum given only as (x, y, σ) is not yet **matchable** between images, since it carries neither an orientation nor a signature.

SIFT, introduced by David Lowe in 1999 [2] and refined in 2004 [1], addresses both problems. The LoG is approximated by an inexpensive **difference of Gaussians**, the computation is organized into a **pyramid**, and each keypoint is given an **orientation** and a **128-dimensional descriptor**. The method is fast, running in milliseconds in OpenCV, and it is remarkably robust.

+ board

Outline of this lecture

1. We show that the **DoG approximates the scale-normalized LoG**, using the heat equation.
2. We define an **octave** as a doubling of σ and s as the number of scales searched within it, then construct the **DoG scale space** and the **Gaussian pyramid**.
3. We detect **keypoints** as extrema of the DoG over 26 neighbors, followed by sub-pixel refinement and the rejection of low-contrast and edge responses.
4. We assign a **dominant orientation** using a 36-bin gradient histogram, which yields a reference angle.
5. We construct the **128-dimensional descriptor** from a 16×16 grid divided into 4×4 blocks of 8 orientations, suitably normalized.
6. We establish which transformations SIFT is invariant to, namely changes of scale, rotation and illumination, and how two sets of descriptors are **matched**.

+ board

Difference of Gaussians

We approximate by a **finite difference** between two nearby scales and :

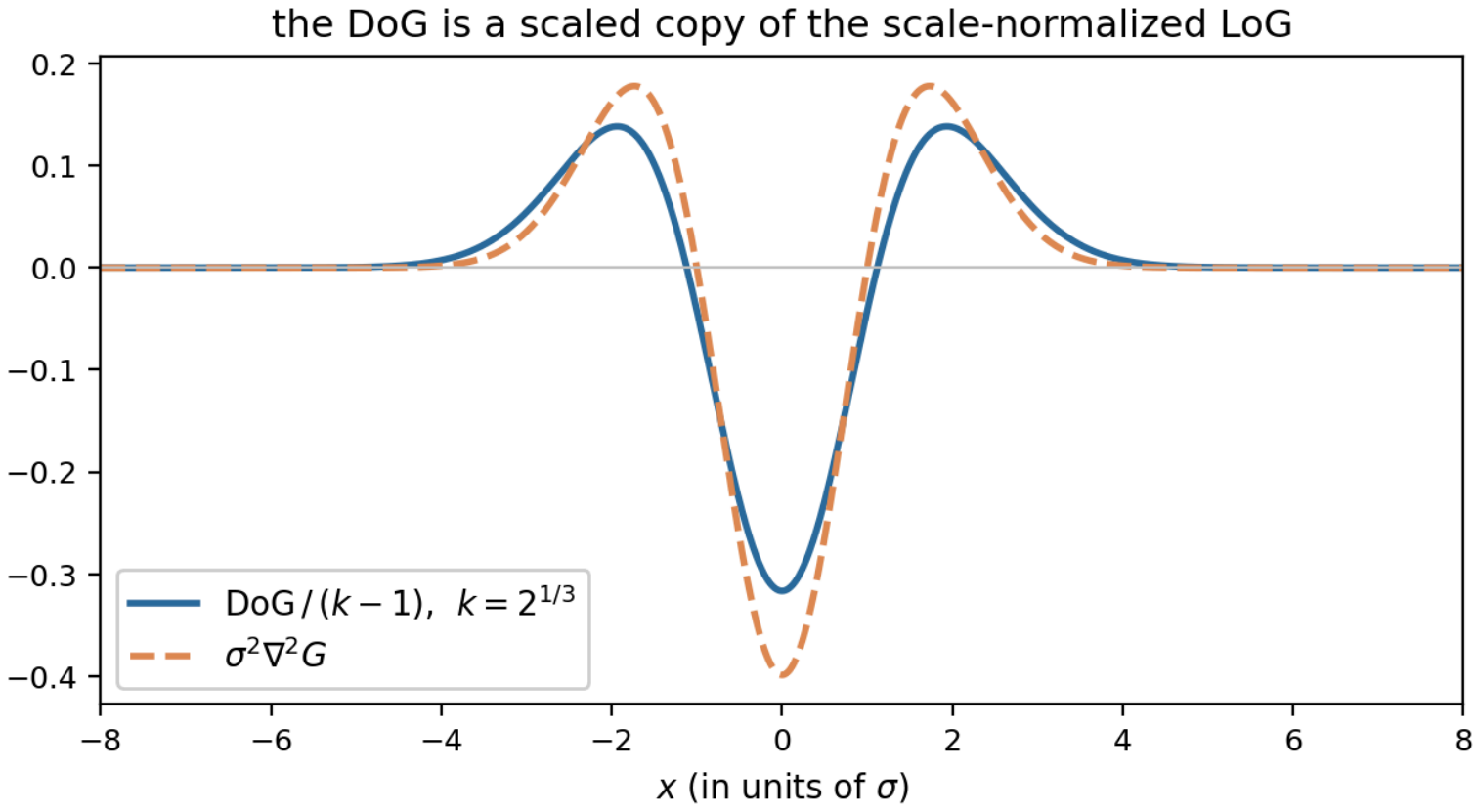
Combining this with gives

The bracketed term is a **difference of Gaussians**, or DoG. The DoG is therefore a scaled copy of the scale-normalized LoG, and the constant factor does not change the positions of the extrema.

The advantage. A DoG requires only **two Gaussian blurs and a subtraction**. No special LoG kernel is needed, and Gaussian blurs are inexpensive and separable.

The qualification. The approximation is exact only in the limit . Lowe reports that even a separation as large as $\sqrt{}$ has almost no effect on the stability of detection, and the value actually used follows from the sampling of scale, being . The value of is empirical.

LoG vs DoG


the DoG is a scaled copy of the scale-normalized LoG
0.2
0.1
0.0
−0.1
−0.2
−0.3
−0.4
−8
−6
−4
−2
0
2
4
6
8
x (in units of σ)
DoG$/(k-1)$, $k = 2^{1/3}$
$\sigma^2 \nabla^2 G$

What is an octave, and what is s?

An octave is one doubling of σ. Inside it we must decide *how finely to sample scale*, and that decision is s:

> **s = the number of scales per octave at which extrema are actually searched.**

Everything else is *forced* by it. Those s scales must climb a factor of two in s equal multiplicative steps, so one step is k with $k^s = 2$:

$$k = 2^{1/s} \qquad s = 3 \;\Rightarrow\; k = 2^{1/3} \approx 1.26$$

s is **chosen**; k is **derived**. Lowe takes $s = 3$ and starts each octave at $\sigma = 1.6$. Both are empirical: repeatability peaks at three scales, and sampling more finds *more* extrema but less stable ones.

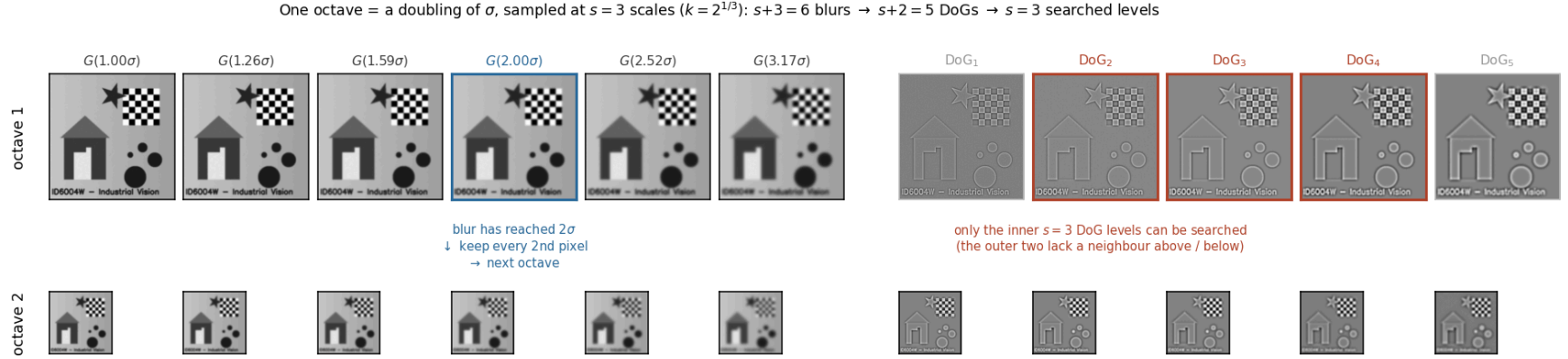
What $s = 3$ actually produces, drawn on the next slide:

	#	scales
blurs $(s+3)$	6	$\sigma,\ 1.26\sigma,\ 1.59\sigma,\ 2\sigma,\ 2.52\sigma,\ 3.17\sigma$
DoGs $(s+2)$	5	one per adjacent pair
searched (s)	**3**	$1.26\sigma,\ 1.59\sigma,\ 2\sigma$

The outer two DoGs are dropped for lacking a neighbor above or below - which is why $s + 3$ blurs are needed and not $s + 1$. Blurring is **incremental**, each level from the one below, and nothing is re-sampled: every image here shares the **same pixel grid**.

+ board

The same thing as a picture



Left, six blurs. $s + 3 = 6$ Gaussians of the *same* image on the *same* grid, at σ through 3.17σ in steps of $k = 1.26$.

Right, five DoGs. Each is the difference of an adjacent pair. Only the **three boxed in red** have a DoG above *and* below, so only those can be searched - that is $s = 3$.

Below, the next octave. The blur marked in blue has reached 2σ; keeping every second pixel of it halves the grid and the whole construction repeats.

+ board

Across octaves: the Gaussian pyramid

Once an octave is complete, the blurred image whose scale has reached 2σ - being s steps up, that is, **two images from the top of the stack** - is **re-sampled by keeping every second pixel of every second row**.

The next octave then begins at the same σ *relative to its new pixel spacing*. The accuracy of sampling relative to σ is therefore **identical in every octave**, while the computation falls by a factor of four.

A large structure, halved in size, is now found at a **small** σ, and a small σ means a small kernel.

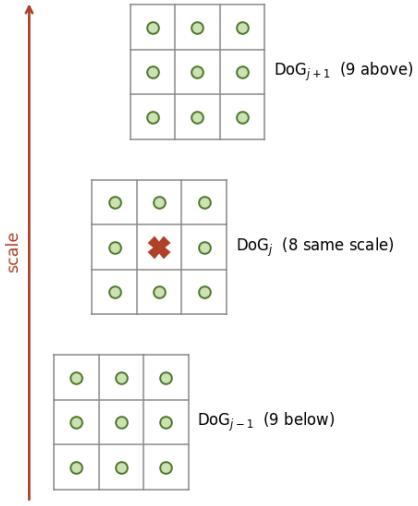
The octaves join up exactly. The last scale searched in an octave is 2σ where the next one begins. So the searched scales form **one unbroken geometric ladder** of ratio k through the whole pyramid, with no gap and no repetition:

$$\underbrace{1.26\sigma,\ 1.59\sigma,\ 2\sigma}_{\text{octave 1}},\ \underbrace{2.52\sigma,\ 3.17\sigma,\ 4\sigma}_{\text{octave 2}},\ \ldots$$

- Down-sampling is safe **precisely because** the image has already been blurred to 2σ: the Gaussian is itself the anti-aliasing filter. Blurring first, then discarding samples, is the correct order.
- A **pyramid** also provides stable coarse-to-fine estimates.
- Octaves continue until the image becomes too small to search, so their number is not fixed but grows like $\log_2$ of the image size.
- Lowe additionally **doubles the input image** by linear interpolation *before* the first octave. Assuming the original carries a blur of $\sigma = 0.5$, the doubled image has $\sigma = 1.0$ on its new grid; this recovers the highest spatial frequencies and yields nearly four times as many stable keypoints.

+ board

Steps 1 and 2: find an extremum, then refine it



Step 1: an extremum over 26 neighbors. Every point of every searchable DoG level is compared with 8 neighbors at its own scale, 9 above and 9 below. It survives only if it is **larger than all of them, or smaller than all of them**.

The two cases are not the same thing. The DoG approximates $\sigma^2 \nabla^2 G$, so a **bright** blob (an intensity peak, negative Laplacian) is a **minimum**, and a **dark** blob is a **maximum**. Both are kept.

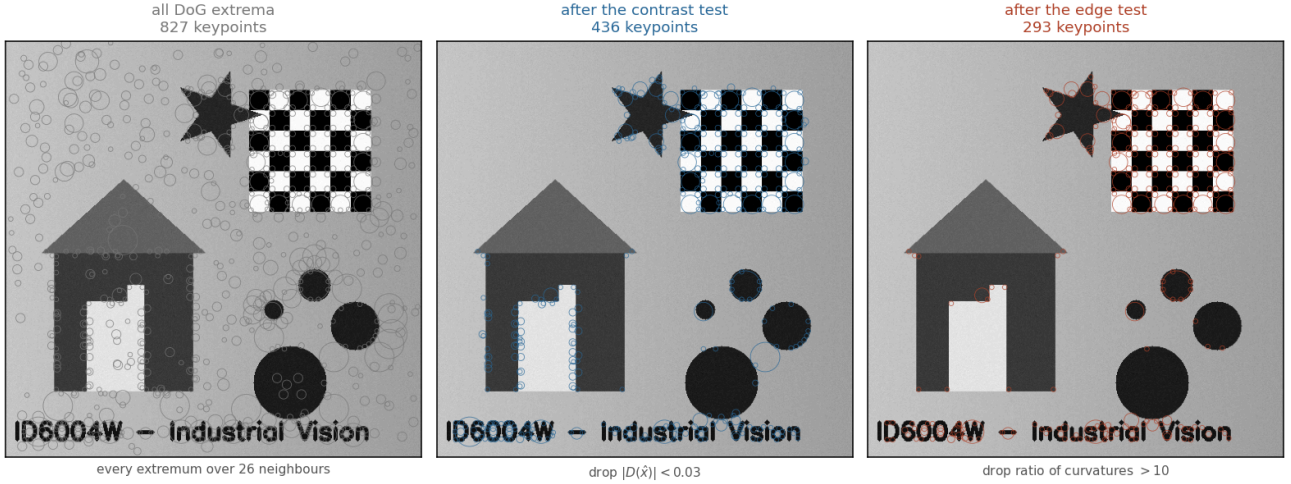
Step 2: refine off the grid. After doing the Taylor expansion, now on $D(x, y, \sigma)$:

$$\hat{\mathbf{x}} = -\mathbf{H}_D^{-1} \nabla D, \qquad \mathbf{H}_D = \frac{\partial^2 D}{\partial \mathbf{x}^2}$$

What is the size of $\mathbf{H}_D$ and ∇D?

$\mathbf{H}_D$ is the 3×3 Hessian in (x, y, σ).

Steps 3 and 4: two cheap tests throw most of them away



Contrast. Evaluate D at the *refined* location and drop what is faint - mostly noise:

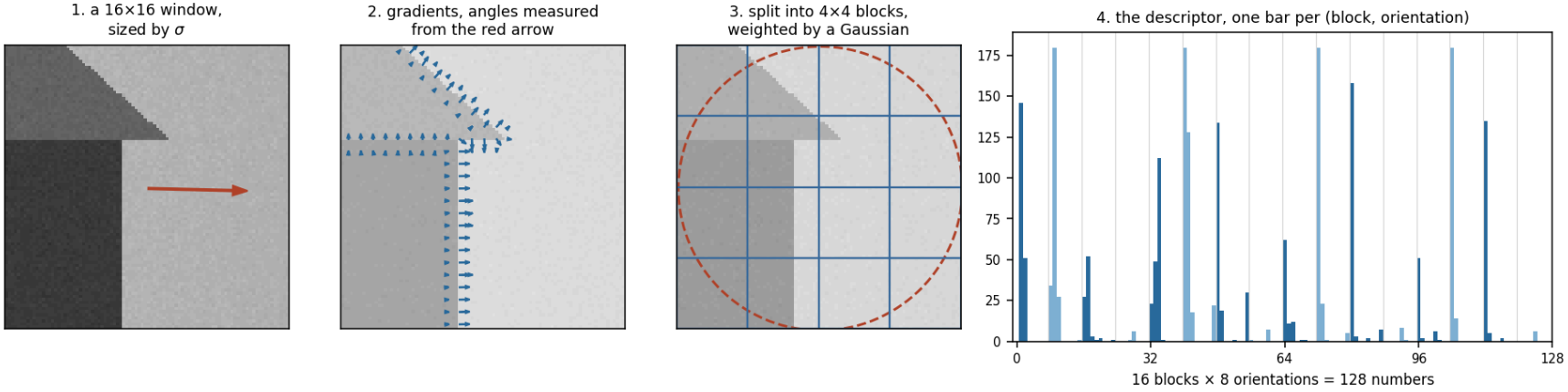
$$D(\hat{\mathbf{x}}) = D + \tfrac{1}{2}\nabla D^{\mathsf{T}}\hat{\mathbf{x}}, \qquad \text{drop } |D(\hat{\mathbf{x}})| < 0.03$$

Edges. Drop points whose curvature is strongly one-sided:

$$\frac{\mathrm{Tr}(\mathbf{H})^2}{\det(\mathbf{H})} < \frac{(r_{\max}+1)^2}{r_{\max}}, \qquad r_{\max} = 10$$

+ board

The 128-dimensional descriptor



1. A 16×16 window, its physical size set by σ, on the blurred image of that scale.

2. Angles measured **from the reference arrow** to obtain rotation invariance.

3. Split into 4×4 blocks; magnitudes weighted by a Gaussian of **half the window width**, and spread into neighboring bins by **trilinear interpolation**.

4. Each block gives an 8-bin histogram. Concatenated **in a fixed order**: $16 \times 8 = \mathbf{128}$.

Building the descriptor, step by step

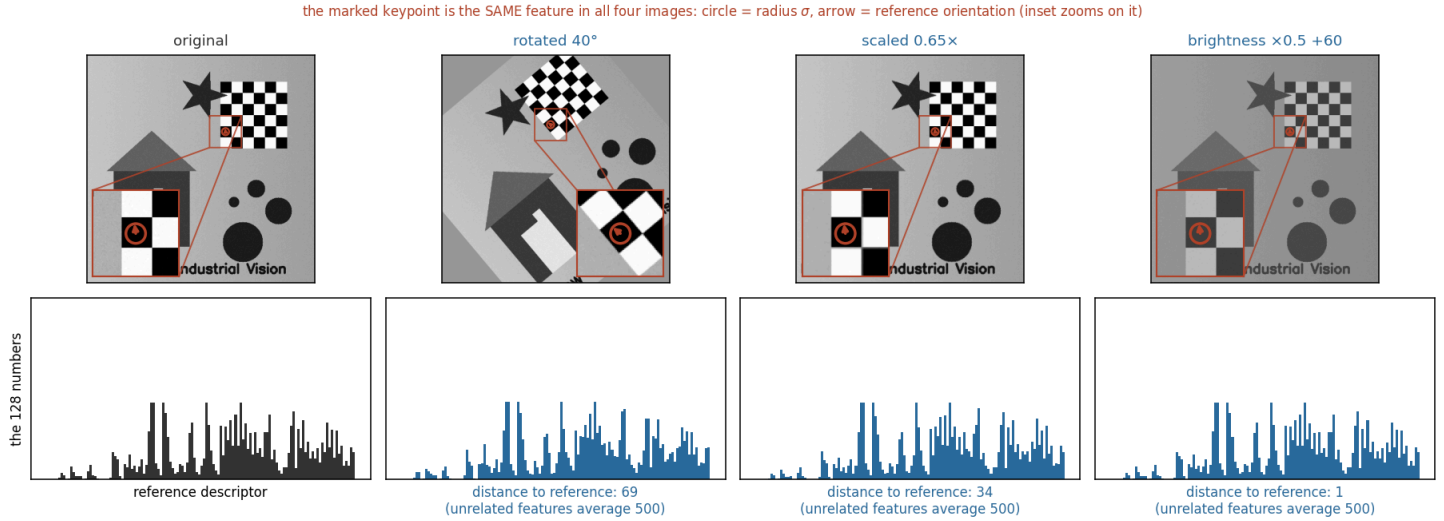
1. **Cut out a patch** around the keypoint, its size measured in units of σ rather than in pixels.
2. **Turn the patch** so that the keypoint's arrow θ points "up", and measure every gradient angle relative to that arrow.
3. **Divide the patch into a** 4×4 **grid** of cells.
4. **In each cell, build an 8-bin histogram** of gradient directions, where each pixel votes with its gradient strength and pixels near the centre count more.
5. **Let votes near a boundary be shared** between neighboring cells and bins, so that a small shift of the image changes the numbers only a little.
6. **Stack the 16 histograms** into $16 \times 8 = 128$ numbers, and **normalize** the vector to unit length.

What each step buys us

- Step 1: **scale** invariance, since a bigger blob gets a bigger patch.
- Step 2: **rotation** invariance, since a rotated image yields the same rotated patch.
- Steps 3 and 4: a **signature** that records *where* edges lie and *which way* they point.
- Step 5: **stability**, since the keypoint is never found at exactly the same pixel twice.
- Step 6: **illumination** invariance, since brightening the image scales every number equally and normalization cancels it.

+ board

Does it actually work? The same corner, four ways



The red circle is the keypoint, its radius σ and its arrow the reference orientation θ; each inset zooms on it, over a window a fixed number of σ wide, so all four insets show the **same physical structure** - the corner of the checkerboard.

Scale ✓ - σ sets the window size, so the same structure is sampled at any magnification.

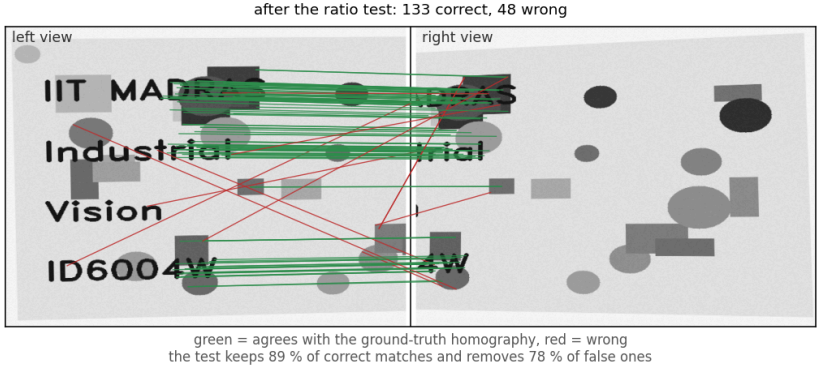
Rotation ✓ - angles are stored relative to θ.

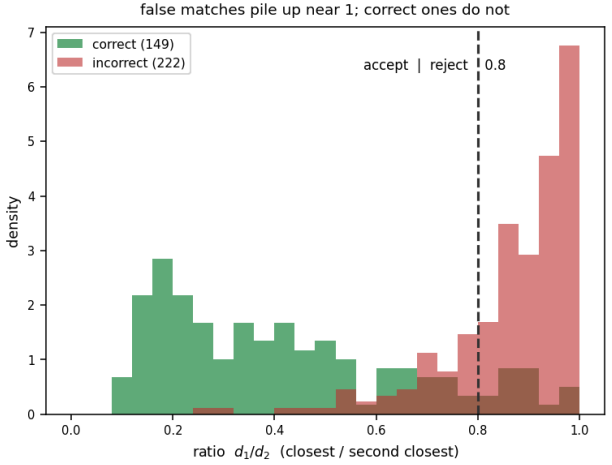
Illumination ✓ - gradients kill an added b; unit-length normalization kills a gain a. Any *affine* lighting change cancels exactly.

The numbers. Distances of 69, 34 and 1 against an average of 485 to unrelated features. That gap is what makes matching possible.

+ board

Matching two sets of descriptors





The problem. Most keypoints have *no* correct counterpart - clutter, occlusion, or they simply were not detected in the other view. A fixed distance threshold cannot separate them, because descriptors differ enormously in how distinctive they are.

Lowe's ratio test. Compare the closest neighbor against the **second closest** and keep the match only if

$$\frac{d_1}{d_2} < 0.8$$

A true match is *distinctly* closer than its best rival; a false one sits in a crowd of equally distant descriptors.

Demonstration of DoG keypoints and matching

image Synthetic scene *ideal step edges, exact corners — the textbook case*

DoG threshold = 0.50 × max `0 keypoints`

DoG extrema (colour = octave scale, radius = √2 σ)

|DoG| at the middle scale

Each keypoint beat all **26 neighbours** (8 same scale + 9 above + 9 below) in the DoG stack. Lower the threshold to admit weaker extrema. This is SIFT's detection stage; orientation + the 128-D descriptor come from the Python demo.

loading opencv.js …

+ board

Demonstration of SIFT features and matching

image pair Boat (zoom + rotation) *zoom and in-plane rotation only — the best-conditioned pair here*

1 · SIFT keypoints | 2 · match descriptors

Each circle is a keypoint: **radius 2σ** shows its scale and the **red tick** its dominant orientation θ. The circles are what the 128-D descriptor is computed from.

running SIFT on both views …

+ board

Summary

- Normalized LoG: $\sigma^2 \nabla^2 g \approx \dfrac{g(k\sigma) - g(\sigma)}{k - 1}$, so that a **DoG** may replace the scale-normalized LoG, at the cost of only two blurs and a subtraction.
- An **octave is a doubling of** σ. The one free choice is s, the number of scales per octave actually **searched**; then $k = 2^{1/s}$ follows. Lowe uses $s = 3, \sigma = 1.6$, and $s + 3$ blurs per octave. At the end of an octave the 2σ image is **down-sampled by two**, so that large structures are found at small σ and cheaply.
- **Keypoints** are extrema of the DoG over 26 neighbors, refined to sub-pixel accuracy by the Taylor and Newton step of L05, then rejected for **low contrast** ($|D| < 0.03$) and for lying on an **edge** ($\mathrm{Tr}^2/\det$ against $r = 10$), giving a triple (x, y, σ).
- The **orientation** is obtained from a magnitude-weighted histogram with 36 bins, which gives a reference angle. An additional keypoint is created for each bin exceeding 80 % of the peak, giving (x, y, σ, θ).
- The **descriptor** is formed from a 16×16 grid divided into 4×4 blocks of 8 orientations, giving **128 dimensions**. The angles are relative to θ, samples are spread by trilinear interpolation, and the vector is normalized to unit length with a contrast cap of 0.2, which makes it robust to changes of scale, rotation and illumination.
- Descriptors are **matched** by nearest neighbor in $\mathbb{R}^{128}$, accepted only when the **ratio** of the closest to the second-closest distance is below 0.8.

+ board

References

1. Lowe, David G. "Distinctive image features from scale-invariant keypoints." International journal of Industrial Vision 60.2 (2004): 91-110.
2. Lowe, David G. "Object recognition from local scale-invariant features." Proceedings of the seventh IEEE international conference on Industrial Vision. Vol. 2. Ieee, 1999.
3. Lindeberg, Tony. "Feature detection with automatic scale selection." International journal of Industrial Vision 30.2 (1998): 79-116.
4. Forsyth, David A., and Jean Ponce. Industrial Vision: a modern approach. prentice hall professional technical reference, 2002. See §5.3.

+ board